Smart Campus Placement Management System

Application Implementation on the Phase 1 Database

Database Systems (BACSE202)

Fall Semester 2026-27

Project Phase 2

Database: smart_campus_placement

MySQL 8.0.46 Community Edition - FastAPI - React - InnoDB - utf8mb4

Submitted by: Prayag Krishna

25BCE2223

Contents

1. Introduction

- 1.1 Background
- 1.2 Objectives of Phase 2
- 1.3 Scope, and what deliberately did not change

2. System Architecture

- 2.1 Overview
- 2.2 Component responsibilities
- 2.3 Why the schema is reflected rather than declared
- 2.4 Technology stack

3. MySQL Integration

- 3.1 Connection
- 3.2 Credential handling
- 3.3 Transactions
- 3.4 Error translation
- 3.5 Health endpoint

4. Demonstration Data

- 4.1 Strategy
- 4.2 Repeatability without destruction
- 4.3 Internal consistency
- 4.4 Volumes loaded
- 4.5 Deliberately awkward cases

5. Backend API

- 5.1 Structure
- 5.2 Endpoints implemented
- 5.3 Response conventions
- 5.4 Input validation

6. Business Logic

- 6.1 The cross-table rules carried over from Phase 1
- 6.2 Further rules enforced
- 6.3 Documented project policies
- 6.4 Eligibility evaluation
- 6.5 Concurrency
- 6.6 Why no trigger was created

7. SQL Queries and Reporting

- 7.1 Reports implemented
- 7.2 Relational features used
- 7.3 No derived value is stored
- 7.4 Avoiding repeated queries

8. Frontend

- 8.1 Approach
- 8.2 Screens implemented

8.3 Data, states and errors

9. Testing and Verification

9.1 Approach

9.2 Result

9.3 Coverage of the required test cases

9.4 Manual verification of the API

9.5 Verification of the write path

9.6 What was not tested

9.7 Defects found and corrected during this phase

10. Implementation Evidence

11. Limitations

12. Conclusion

13. Further Work

Appendix A. API Endpoint Reference

Appendix B. Selected Source Listings

B.1 Eligibility evaluation, from backend/app/rules.py

B.2 The placement rule, from backend/app/rules.py

B.3 A reporting query, from backend/app/routers/reports.py

B.4 Transaction scope, from backend/app/db.py

B.5 Running the system

1. Introduction

1.1 Background

Phase 1 of this project produced a normalised relational database for a college placement cell. It delivered the requirement analysis, the EER model, the relational mapping, a 1NF to 3NF normalisation analysis carried out against the actual candidate keys, and the executable DDL that creates thirteen tables in MySQL 8.0.

That phase deliberately stopped at the schema. A MySQL CHECK constraint may read only the columns of the row currently being written. It cannot consult another table, another row, or a subquery. Several rules that the placement cell genuinely needs therefore could not be expressed in DDL at all. Phase 1 recorded them rather than faking them, sorting every constraint into three groups:

- Constraints the DDL enforces by itself: primary keys, foreign keys, UNIQUE, NOT NULL, DEFAULT, ENUM domains, and CHECK constraints that read only the row being written.
- Cross-table rules: ten rules, each of which must read another table or another row, and so requires either a trigger or application logic. They are listed in full in section 6.1.
- Application and business logic: eligibility screening, skill matching, round progression and placement statistics.

Phase 1 contained no sample data, no triggers, no views and no application code. Those were listed as planned work, not as delivered work.

1.2 Objectives of Phase 2

Phase 2 turns the Phase 1 schema into a system that actually runs. Its objectives are:

1. Establish real connectivity between an application and the existing MySQL database.
2. Populate the database with realistic, internally consistent demonstration data.
3. Expose the schema through a documented REST API.
4. Provide a usable browser interface for the placement cell.
5. Implement the cross-table rules and the business logic in the application layer.
6. Implement relational queries and reports over the live data.
7. Handle errors cleanly and keep multi-statement operations transactional.
8. Verify the result with automated tests that run against the real database.

1.3 Scope, and what deliberately did not change

The Phase 1 schema is treated as locked. Phase 2 added no table, no column, no primary key, no foreign key, no UNIQUE constraint, no CHECK constraint and no ENUM value. Nothing was renamed, relaxed or removed. No trigger and no view was created.

This was verified against the live server after the implementation was complete:

Property	Phase 1	After Phase 2
Base tables	13	13
Foreign key constraints	14	14
CHECK constraints	21	21
Views	0	0
Triggers	0	0
Non-InnoDB tables	0	0

The design choice that makes this guarantee structural rather than merely careful is described in section 2.3: the backend reflects the live tables at runtime instead of declaring its own copy of them.

> The Phase 1 report, its editable source and all Phase 1 SQL scripts and evidence images are unchanged by this phase.

2. System Architecture

2.1 Overview

The system is a three tier application. The browser holds no credential and never addresses the database. It calls a relative path, which the development server proxies to the API, which is the only component that speaks to MySQL.

```
Browser                 Dev server            API server            Database
React 18 + TypeScript   Vite  :5173           FastAPI  :8000        MySQL 8.0.46
                                                                     smart_campus_placement

  fetch('/api/...')  ->  proxy /api  ------>  routers
                                              service rules
                                              SQLAlchemy 2.0 Core  ->  PyMySQL  ->  InnoDB
  render  <-------------  JSON  <-----------  response models
```

2.2 Component responsibilities

Layer	Location	Responsibility
Configuration	backend/app/config.py	Reads the environment and builds the connection URL. Holds no credential.
Database access	backend/app/db.py	Engine, lazy schema reflection, one transaction per request, query helpers.
Validation	backend/app/schemas.py	Pydantic models mirroring the Phase 1 CHECK ranges and ENUM domains.
Business rules	backend/app/rules.py	The cross-table rules of section 6.1 and the project policies of section 6.3.
Error translation	backend/app/errors.py	MySQL error numbers to HTTP status codes. No trace reaches a client.
HTTP routes	backend/app/routers/	meta, students, drives, workflow and reports.
Demo data	backend/seed.py	Idempotent seed keyed on natural keys, plus a scoped reset.
Tests	backend/tests/	Integration tests against the real database.
Frontend	frontend/src/	React 18 with TypeScript, built by Vite. Fifteen screens.

2.3 Why the schema is reflected rather than declared

A conventional object relational mapping would restate every table as a Python class. Those classes then become a second, independent description of the schema, and the two descriptions can disagree without anybody noticing.

This backend instead calls MetaData.reflect() once, on first use, and reads the live table definitions directly from the running database. There is no model file that could contradict 02_create_tables.sql. If a column were renamed in the database, the application would fail immediately and visibly rather than writing against a stale definition. The Phase 1 DDL remains the single authority on the schema.

Reflection is deferred until the first query, so the process still starts and the health endpoint still answers when MySQL is unavailable. The expected set of thirteen table names is held as a constant and compared against what was actually reflected, so a missing table is reported by name.

2.4 Technology stack

Layer	Technology	Version
Database	MySQL Community Edition	8.0.46-0ubuntu0.24.04.4
Storage engine	InnoDB	-
Character set	utf8mb4 / utf8mb4_0900_ai_ci	-
Backend language	Python	3.12.3
Web framework	FastAPI	0.143.1
ASGI server	Uvicorn	0.54.0
Database toolkit	SQLAlchemy Core	2.0.54
MySQL driver	PyMySQL	1.2.3
Validation	Pydantic	2.14.1
Frontend library	React	18.3.1
Frontend language	TypeScript	5.6.3
Build tool	Vite	5.4.11
Test runner	pytest	9.1.1

Everything runs locally. No cloud database, no hosted service and no paid dependency is involved, which matches the constraint carried over from Phase 1.

3. MySQL Integration

3.1 Connection

The application connects to the database created in Phase 1. Nothing is recreated.

Item	Value
Server	MySQL 8.0.46-0ubuntu0.24.04.4
Database	smart_campus_placement
Engine and character set	InnoDB, utf8mb4
Account	campus_app@localhost, database scoped, no global privileges
Driver	PyMySQL 1.2.3 through SQLAlchemy 2.0.54
Tables reflected at runtime	13 of 13

The account carries GRANT USAGE ON *.* with no global rights, plus ALL PRIVILEGES on the single project database. It holds no GRANT OPTION, no SUPER and no FILE privilege. This least privilege arrangement was established in Phase 1 and is unchanged.

3.2 Credential handling

No password appears in source code, in a committed file, in a log line, in an API response or in the compiled frontend bundle. This was verified by scanning every file in the repository and every file published to the remote.

Configuration is read from the environment, or from backend/.env, which git ignores. The committed backend/.env.example carries an empty DB_PASSWORD placeholder.

When DB_PASSWORD is left empty, the application passes read_default_file to the MySQL driver. The driver then reads the local client option file in exactly the way the mysql command line client does. The application itself never opens, copies, prints or commits that file. An explicit DB_PASSWORD environment variable may be used instead.

The browser never receives a credential of any kind. The frontend requests a relative /api path, and the development server proxies it, so the backend host is not even present in the client bundle.

3.3 Transactions

A FastAPI dependency opens one connection and one transaction per request, and the route handler runs inside it. If the handler raises, the whole request is rolled back; if it returns, the transaction commits. Atomicity is therefore the default rather than something each handler must remember.

The clearest beneficiary is offer issue. Creating an offer also advances the application to the Selected status. Both writes share the one transaction, so the offer and the application status can never disagree.

3.4 Error translation

MySQL reports failures as numbered errors, and SQLAlchemy wraps them in different exception classes depending on when they are raised. The backend maps the error number in one shared table, used by every handler, and converts it to an HTTP status code with a readable message that names the violated constraint.

MySQL error	Meaning	HTTP status
1062	Duplicate entry for a UNIQUE key	409 Conflict
1452	Referenced parent row does not exist	409 Conflict
1451	Row is still referenced by a child	409 Conflict

MySQL error	Meaning	HTTP status
3819	CHECK constraint violated	422 Unprocessable
1048	Required column was null	422 Unprocessable
1264	Value out of range for the column	422 Unprocessable
1265	Value is not a member of the ENUM	422 Unprocessable
Connection failure	Database unreachable	503 Service Unavailable

No exception trace, SQL statement or connection string is ever returned to a client. An automated test asserts that an error response contains none of the strings 'traceback', 'insert into', 'pymysql', 'password' or 'sqlalchemy'.

3.5 Health endpoint

GET /api/health reports API liveness and database reachability as two separate fields, so a database outage can be distinguished from the service being down altogether.


```
{ "api": "ok",
  "database": "ok",
  "database_name": "smart_campus_placement",
  "mysql_version": "8.0.46-0ubuntu0.24.04.4",
  "table_count": 13,
  "detail": null }
```


If any of the thirteen Phase 1 tables were absent, the detail field would name the missing ones rather than merely reporting a generic failure.

4. Demonstration Data

4.1 Strategy

The system has to be demonstrable immediately after setup, so backend/seed.py populates the database with a realistic data set. Every company, student, electronic mail address and telephone number in it is fictional, and the electronic mail domains use the reserved .example suffix. No real student personal data is used.

4.2 Repeatability without destruction

Each row is matched on its natural key rather than on a generated identifier: dept_code, skill_name, company_name, reg_no, the drive's composite (company_id, job_title, drive_date), and the composite primary keys of the four bridge relations. Running the script a second time therefore updates the existing rows instead of duplicating them.

The script contains no DROP TABLE and no TRUNCATE, and it never disables foreign key checking. It was executed three times in succession and produced identical row counts on each run.

A scoped reset is also provided. seed.py --reset deletes only the departments, skills, companies and students that the script itself owns, children first. If a row created by somebody else references demonstration data, a RESTRICT foreign key stops the delete and the script reports the obstruction rather than removing the other party's work. seed.py --summary prints current row counts without writing anything.

4.3 Internal consistency

The seed obeys the same rules the API enforces, so the demonstration data could have been produced through the interface:

- An application is generated only for a student who genuinely satisfies that drive's criteria, including the department restriction.
- An application date never falls after the drive's application deadline.
- An offer is issued only to an applicant who cleared every interview round.
- A placement record is written only against an accepted offer whose expected joining date has already passed.

4.4 Volumes loaded

Table	Rows	Table	Rows
department	6	drive_eligible_department	34
skill	20	drive_required_skill	26
company	10	application	64
student	40	interview	156
student_skill	207	offer	11
job_drive	14	placement_record	5
eligibility_criteria	13		

4.5 Deliberately awkward cases

A data set in which everything is populated proves very little. The seed therefore includes the cases that exercise the harder parts of the design:

- One drive has no eligibility_criteria row at all, which exercises the optional one-to-one relationship and the missing criteria policy in section 6.3.
- One drive has no drive_eligible_department rows, which exercises the missing department policy in section 6.3.
- Drives exist in all five states: Upcoming, Open, Closed, Completed and Cancelled.
- Applications exist in five of the six statuses that the workflow can reach.
- Offers exist in all five statuses: Offered, Accepted, Declined, Expired and Revoked.
- Some accepted offers have no verified placement record yet, which is a genuine operational state rather than an inconsistency.

The last case is the one that justifies keeping offer and placement_record as separate relations. An offer records what a company proposed. A placement record is a joining that somebody has verified. Eleven offers produced only five verified placements, and the report named accepted-offers-not-verified lists the difference.

5. Backend API

5.1 Structure

The backend is organised so that each concern lives in one place. Routers contain no business rule, and the rules module contains no HTTP detail. The whole backend, including tests and the seed script, is approximately 4,310 lines of Python.

5.2 Endpoints implemented

Fifty-two endpoints are implemented. Interactive documentation is generated from the code itself and served at /docs, so it cannot describe an endpoint that does not exist. The complete list appears in Appendix A.

Area	Count	Coverage
Health and dashboard	2	Liveness, database state, aggregate totals
Students and departments	10	Search, six filters, detail, create, update, skill assignment
Skills catalogue	2	List with category filter, detail
Companies	4	List with drive counts, detail, create, update
Job drives	14	Search, detail, criteria, eligible departments and skills, eligibility evaluation
Applications	5	Submit with full rule checking, list, detail, status change, re-check
Interviews	4	Schedule a round, list, detail, record the result
Offers	5	Issue, list, detail, update status, offer for an application
Placements	4	Record a verified joining, list, detail, update
Reports	2	Report index and a named report runner

5.3 Response conventions

Every list endpoint returns the same envelope, where total is the count before limit and offset are applied:

```
{ "items": [ ... ], "total": 64, "limit": 25, "offset": 0 }
```

Errors return a detail string. A business rule rejection additionally returns a reasons array, so the interface can tell the user precisely what failed instead of showing a generic message:

```
{ "detail": "Student is not eligible to apply to this drive.",
  "reasons": [ "CGPA 6.10 is below the required 8.00.",
               "Student has 3 active backlogs, the drive allows at most 2." ] }
```

Situation	Status
Successful read or update	200
Resource created	201
Resource deleted, no body	204
Referenced record does not exist	404
Business rule violated, or a UNIQUE or FOREIGN KEY constraint rejected the write	409
Input failed validation, or a database CHECK rejected the value	422
Database unreachable	503

5.4 Input validation

Pydantic request models restate the Phase 1 domain constraints, so an invalid request is rejected before it reaches MySQL and the caller receives a precise field level message. CGPA is bounded to the range 0 to 10, percentages to 0 to 100, CTC must exceed zero, batch year is bounded to 2000 to 2100, academic year must match the pattern YYYY-YYYY, and every ENUM column is typed as a literal union of its permitted values. Unknown fields are rejected rather than silently ignored.

This duplicates the database constraints on purpose. The database remains the final authority, and every one of these rules is still enforced there; the application layer exists to produce a better error message, not to replace the constraint.

6. Business Logic

6.1 The cross-table rules carried over from Phase 1

Phase 1 documented ten rules that no CHECK constraint can express, because each one has to read another table or another row. Phase 2 implements nine of the ten in the service layer. Because they sit in the service layer rather than in the interface, they apply to every caller, including a client that addresses the API directly. Each rule is named and stated in full below, and the report uses those descriptions throughout rather than any abbreviation.

Rule	What it requires	Status
Placement needs an accepted offer	A placement record may exist only against an offer whose status is Accepted	Implemented, tested
Application deadline	An application cannot be submitted after the drive's application deadline	Implemented, tested
Academic criteria	The applicant must satisfy the drive's CGPA, backlog, percentage and batch year requirements	Implemented, tested
Eligible department	The applicant's department must appear in the drive's eligible department list	Implemented, tested
Placed students excluded	When allow_placed_students is FALSE, an already placed applicant is blocked	Implemented, tested
Offer selects the application	Issuing an offer sets that application's status to Selected	Implemented, tested
Interview follows the application	An interview cannot be scheduled before its application was made	Implemented, tested
Drive status moves forward only	drive_status may not move backwards, for example from Completed to Open	Implemented, tested
Employee code unique per company	company_employee_code must be unique within one company	NOT IMPLEMENTED
One offer per application	An application may have at most one offer	Implemented, tested

> One rule of the ten is outstanding: company_employee_code is not checked for uniqueness within a company. That uniqueness is scoped to a company, which the present schema does not express, and no Phase 2 screen depends on it. It is recorded here and in section 11 as incomplete rather than claimed as done.

6.2 Further rules enforced

Beyond the ten rules above, the service layer also enforces the following. Each is backed by a database constraint where one exists, so the application check improves the error message without being the only defence. The constraint name is given in brackets.

- A student may not apply twice to the same drive (uq_application_student_drive).
- An offer may not have more than one placement record (uq_placement_offer).
- A round number is unique within an application (uq_interview_app_round).
- An interview score may not exceed its maximum score (chk_interview_score_max).
- A score may not be recorded without a maximum score.
- An offline interview round must name a venue (chk_interview_venue).
- An offer's expected joining date may not precede its offer date (chk_offer_dates).

- A drive's application deadline must fall on or before its drive date (chk_job_drive_dates).
- An accepted offer that already has a verified placement record cannot be downgraded to another status.

6.3 Documented project policies

Phase 1 left several questions genuinely open. Deciding them silently inside the code would have been the worst outcome, so each is stated explicitly here, reported through the API, and shown in the interface.

Policy 1. A drive with no eligibility criteria has no academic gate

eligibility_criteria is an optional one-to-one dependent entity whose drive_id is both its primary key and its foreign key. A drive may therefore legitimately have no row. Such a drive is reported with criteria_defined set to false.

It is deliberately not treated as though every cut-off were zero. 'No academic requirement has been defined' and 'the academic requirement is zero' are different facts, and a caller must be able to tell them apart. The deadline rule and the department rule still apply to such a drive. The drive detail screen states this condition in plain language rather than leaving the section blank, as Figure 7 shows.

Policy 2. A drive with no eligible departments is open to all departments

While drive_eligible_department holds no row for a drive, every department may apply. Once at least one row exists, the drive is restricted to the listed departments only.

Policy 3. What counts as being already placed

A student counts as placed when they hold an offer with status Accepted, or a verified placement_record. This is the condition that allow_placed_students = FALSE blocks.

Policy 4. Applications are accepted only while the drive is Open

A drive that is Upcoming, Closed, Completed or Cancelled does not accept applications, even if its deadline has not passed.

Policy 5. Drive status moves forward only

The permitted transitions are served from the same constant the API enforces, at GET /api/drive-status-transitions, so the documented policy and the enforced policy cannot diverge.

```
Upcoming   ->  Open, Cancelled
Open       ->  Closed, Cancelled
Closed     ->  Completed, Cancelled
Completed  ->  (terminal)
Cancelled  ->  (terminal)
```

6.4 Eligibility evaluation

Eligibility is the most substantial rule in the system, and it is evaluated in two complementary ways.

For a single student against a single drive, the evaluator returns not merely a boolean but every individual check, with the required value and the actual value side by side, together with human readable reasons for any failure. The interface renders this directly, which is shown in Figure 8. The same function is what POST /api/applications calls before it writes anything, so the explanation the user reads is produced by the identical code path that performs the rejection.

For a whole cohort against one drive, a separate endpoint pushes the criteria into the WHERE clause of a single statement and filters the student table in the database. This avoids issuing one query per student, which calling the per student evaluator in a loop would require. An automated test asserts that the two approaches agree.

6.5 Concurrency

Where two simultaneous requests could both pass an application level check, the database constraint is the real guard. uq_application_student_drive, uq_offer_application, uq_placement_offer and uq_interview_app_round each make the corresponding race impossible at the storage engine level. The resulting error number 1062 is then translated into a 409 response that names the constraint. The application level check exists to produce a helpful message first, not to be the only defence.

6.6 Why no trigger was created

The ten rules in section 6.1 could have been written as triggers. They were not, for three reasons. First, the assignment specification does not require them. Second, a trigger raises SIGNAL with a message that is difficult to present usefully to a user, whereas the service layer can return a structured list of reasons. Third, Phase 1 already recorded the limitation that a MySQL trigger validates at write time only, cannot be deferred to the end of a transaction, and cannot retroactively validate rows that already exist; it is therefore not the formal invariant that a FOREIGN KEY is.

Creating triggers would also have modified the Phase 1 database objects, which this phase deliberately avoids.

7. SQL Queries and Reporting

7.1 Reports implemented

Thirteen named queries are served from GET /api/reports/{name}, and GET /api/reports lists them with their descriptions. Each is a fixed SQL statement; only LIMIT and OFFSET are bound, and they are bound as parameters, so no part of any statement is constructed from user input.

Report	Question it answers
applications-by-company	Applications, offers and acceptances per company and drive
interview-progress	Rounds completed, passed, failed and pending per application, with average score
offers-by-status	Offer counts with average, minimum and maximum CTC
placements-by-batch	Verified placements grouped by student batch year
placements-by-department	Placement count and placement rate per department
placements-by-company	Hires and average offered CTC per company
students-without-applications	Students who have not applied to any drive
applications-without-interviews	Applications with no round scheduled
drives-without-applications	Drives that attracted no applicant
accepted-offers-not-verified	Accepted offers still awaiting a verified joining
skill-demand	Skills ranked by drive demand against student supply
top-offers	Highest offers with the student and company behind each
application-funnel	The placement pipeline counted stage by stage

7.2 Relational features used

Taken together the reports exercise inner joins, left outer joins, GROUP BY, HAVING, COUNT(DISTINCT ...), conditional aggregation with CASE inside an aggregate, NULLIF to avoid division by zero when a department has no students, correlated NOT IN subqueries and UNION ALL. The eligible student query additionally uses a NOT IN subquery against a derived set of already placed students.

7.3 No derived value is stored

This is the point at which the Phase 1 normalisation decisions are either honoured or quietly abandoned. Phase 1 deliberately did not store is_placed, age, total_applicants, students_hired or current_round, because each is functionally determined by other data and storing it would have introduced an update anomaly.

Phase 2 computes every one of them at read time with SQL aggregation. Nothing is cached in a base table and no summary column was added. An automated test compares the dashboard totals against a direct COUNT(*) on each table, and a second test inserts a row and asserts that the corresponding total increases, so the figures are demonstrably derived rather than stored.

No view was created either, so the Phase 1 object list is unchanged.

7.4 Avoiding repeated queries

A common failing in a data driven application is to fetch a list of items and then issue one further query for each item, so that displaying fifty rows costs fifty-one statements. Three measures prevent that here. The dashboard issues a small fixed set of aggregate statements rather than one per entity. List endpoints join their related tables within a single statement, so a page of applications arrives with its student, drive and company data

already attached. Cohort eligibility is evaluated as one filtered query rather than one query per student.

8. Frontend

8.1 Approach

The interface is a placement office administration dashboard rather than a marketing page. It favours dense, readable tables, explicit filters and clear status badges. There are no decorative animations and no landing page concealing the application.

It is built with React 18 and TypeScript, bundled by Vite, and comprises approximately 2,882 lines across fifteen screens. Colours are declared once as custom properties and redefined for dark mode. The layout is responsive and collapses to a menu button below 860 pixels of width.

Repetition is avoided by sharing three components: one generic data table driven by a column specification, one status badge that maps every ENUM value in the schema to a colour, and one modal form driven by a field specification. Every creation and editing screen in the system uses that single form component.

8.2 Screens implemented

Screen	Route	Function
Dashboard	/	Aggregate totals, status breakdowns, pipeline funnel, database state
Students	/students	Search and six filters, pagination, registration form
Student detail	/students/:id	Record, skills with add, edit and remove, applications, outcome
Companies	/companies	List with drive counts, search, creation form
Company detail	/companies/:id	Contact details and the company's drives
Job drives	/drives	Six filters, pagination, creation form
Drive detail	/drives/:id	Criteria, eligible departments, required skills, eligible students, applications, status transitions
Eligibility check	/drives/:id/check/:studentId	Every rule with required against actual, plus skill coverage
Skills	/skills	Catalogue with drive demand against student supply
Applications	/applications	Filters, pagination, submission form
Application detail	/applications/:id	Interview rounds, offer, placement, status actions
Interviews	/interviews	All rounds across applications, with filters
Offers	/offers	Filters including accepted but not yet verified
Verified placements	/placements	Filters by academic year, department and batch
Reports	/reports	Runs any of the thirteen named queries

8.3 Data, states and errors

Every screen reads live API data. There is no hardcoded array standing in for database rows anywhere in the frontend, and every form submits to the API and persists to MySQL.

Loading, empty, success and error states are handled on each screen. When a write is rejected, the form displays the backend's own reasons array rather than a generic failure message, so a student blocked by a CGPA cut-off sees exactly which cut-off blocked them.

There is no authentication. The assignment does not require it, and a login screen that checked nothing would misrepresent the system's security. This is recorded as a limitation in section 11 rather than presented as a feature.

9. Testing and Verification

9.1 Approach

The tests run against the real MySQL database, not against an in-memory substitute. Substituting SQLite would have tested a different engine with different constraint semantics and would have proved nothing about the Phase 1 schema, which is the thing under test.

Isolation is achieved without a throwaway database. Each test opens one connection and one transaction, and the FastAPI dependency that normally supplies a connection is overridden so the route handler receives that same connection. At the end of the test the transaction is rolled back. Every write the test made therefore disappears.

Nothing is dropped, nothing is truncated, and no foreign key check is disabled at any point. The seeded row counts were recorded before and after the suite and were identical, which is the evidence that the demonstration data was not disturbed.

9.2 Result

> 67 tests passed. 0 failed. Execution time 0.96 seconds.

The single warning emitted is a Starlette deprecation notice about the test client library and is not a failure. The complete per test listing is held in docs/phase-2/TEST_RESULTS.md.

9.3 Coverage of the required test cases

Required case	Test	Result
Database connection	test_database_connection_succeeds	Passed
Student retrieval	test_get_student_detail, test_list_students_is_paginated	Passed
Job drive retrieval	test_list_and_get_job_drive	Passed
Valid application submission	test_valid_application_submission	Passed
Duplicate application rejected	test_duplicate_application_rejected	Passed
Ineligible student rejected	three tests covering CGPA, backlogs and batch year	Passed
Application after deadline rejected	test_application_after_deadline_rejected	Passed
Invalid department rejected	two tests, ineligible and non-existent	Passed
Invalid CGPA rejected	test_invalid_cgpa_rejected	Passed
Invalid interview score rejected	test_interview_score_above_max_score_rejected	Passed
Duplicate interview round rejected	test_duplicate_interview_round_rejected	Passed
Duplicate offer rejected	test_duplicate_offer_rejected	Passed
Placement against non-accepted offer rejected	test_placement_against_non_accepted_offer_rejected	Passed
Duplicate placement rejected	test_duplicate_placement_rejected	Passed
Dashboard aggregation correct	two tests, direct comparison and reaction to a new row	Passed

The suite additionally covers the missing criteria and missing department policies of section 6.3, the drive status transition policy in both directions, the student skill composite key lifecycle, all thirteen reports, response shape conventions, and the assertion that error responses leak no internal detail.

9.4 Manual verification of the API

Beyond the automated suite, every endpoint was called against the running server. Twenty-five base endpoints, thirteen report endpoints, twelve filter and pagination combinations and four deliberate not-found requests were exercised, and all returned the expected status code. Representative results:

Request	Status	Result
GET /api/students	200	total 40
GET /api/drives	200	total 14
GET /api/applications	200	total 64
GET /api/interviews	200	total 156
GET /api/offers	200	total 11
GET /api/placements	200	total 5
GET /api/students?min_cgpa=8&max_backlogs=0	200	total 6
GET /api/drives?drive_status=Open	200	total 7
GET /api/offers?unverified_only=true	200	total 1
GET /api/reports/interview-progress	200	51 rows
GET /api/reports/students-without-applications	200	9 rows
GET /api/students/99999	404	not found
GET /api/reports/not-a-report	404	not found

9.5 Verification of the write path

The automated suite rolls every write back, which proves the logic but not persistence. A separate verification therefore wrote a row along the exact network path the browser's own form uses and confirmed it at each stage.

Step	Outcome
POST /api/companies through the development proxy on port 5173	HTTP 201, company_id 231
Row present in MySQL, queried with the command line client	Returned, created_at 2026-10-14 15:17:58
Row visible in the rendered interface	Yes, see Figure 16, which reads 'Showing 1-11 of 11'
Duplicate name submitted again	HTTP 409, 'Violated constraint: company.uq_company_name'
Verification row deleted afterwards	Company count returned to 10

The verification row was removed so that the committed demonstration data matches the seed exactly.

9.6 What was not tested

The following are stated plainly rather than left for a reader to assume.

Claim	Status
Backend starts and serves	Tested successfully
Frontend starts and serves	Tested successfully
Database connectivity	Tested successfully
Seed script works and repeats safely	Tested successfully
Scoped reset works	Tested successfully

Claim	Status
All 52 endpoints return expected responses	Tested successfully
Database writes persist	Tested successfully
Constraints reject invalid data	Tested successfully, with real MySQL error numbers
Frontend displays actual database results	Tested successfully, all fifteen routes rendered
Automated tests pass	Tested successfully, 67 of 67
Nine of the ten cross-table rules in section 6.1	Implemented and tested
The tenth rule, employee code unique per company	NOT IMPLEMENTED
React event handlers driven by an automated browser	NOT TESTED
Behaviour under concurrent load	NOT TESTED
Behaviour at large data volumes	NOT TESTED

On the third of these: no Playwright, Cypress or Selenium suite exists, so no automated process clicks the interface's buttons. The endpoints behind every form are covered by the automated suite, and one write was verified through the browser's own network path as described in section 9.5, but the event handlers themselves were exercised only by hand. A browser end to end suite is the first item of further work in section 12.

9.7 Defects found and corrected during this phase

Four defects were found by the verification process itself. They are recorded because the corrections are part of the work.

Defect	Cause	Correction
A CHECK violation returned 503 'database not available' instead of a client error	MySQL raises error 3819 through a different SQLAlchemy exception class than expected, so it reached the unreachable-server branch	The error number to status mapping was moved into one shared table used by every handler
Navigation links laid out horizontally and overflowed the sidebar	Each navigation group was a block container whose anchors defaulted to inline layout	Each group was made a column, and the anchors made block level
Badges displayed 'Intermediate' for an online interview and 'Pass' for a mandatory skill	Unrelated status words were reused to borrow badge colours	The real words were added to the colour map and the correct labels displayed
Changing a drive date left stale demonstration rows behind	drive_date is part of the drive's natural key, so altering it created new rows instead of updating existing ones	The scoped reset was used and the data reseeded; the behaviour is now documented

10. Implementation Evidence

Every image in this section was captured from the running application against the live smart_campus_placement database. None is a mock-up, a drawing or an edited image. The figures appear in the order in which a user would encounter the corresponding screens.

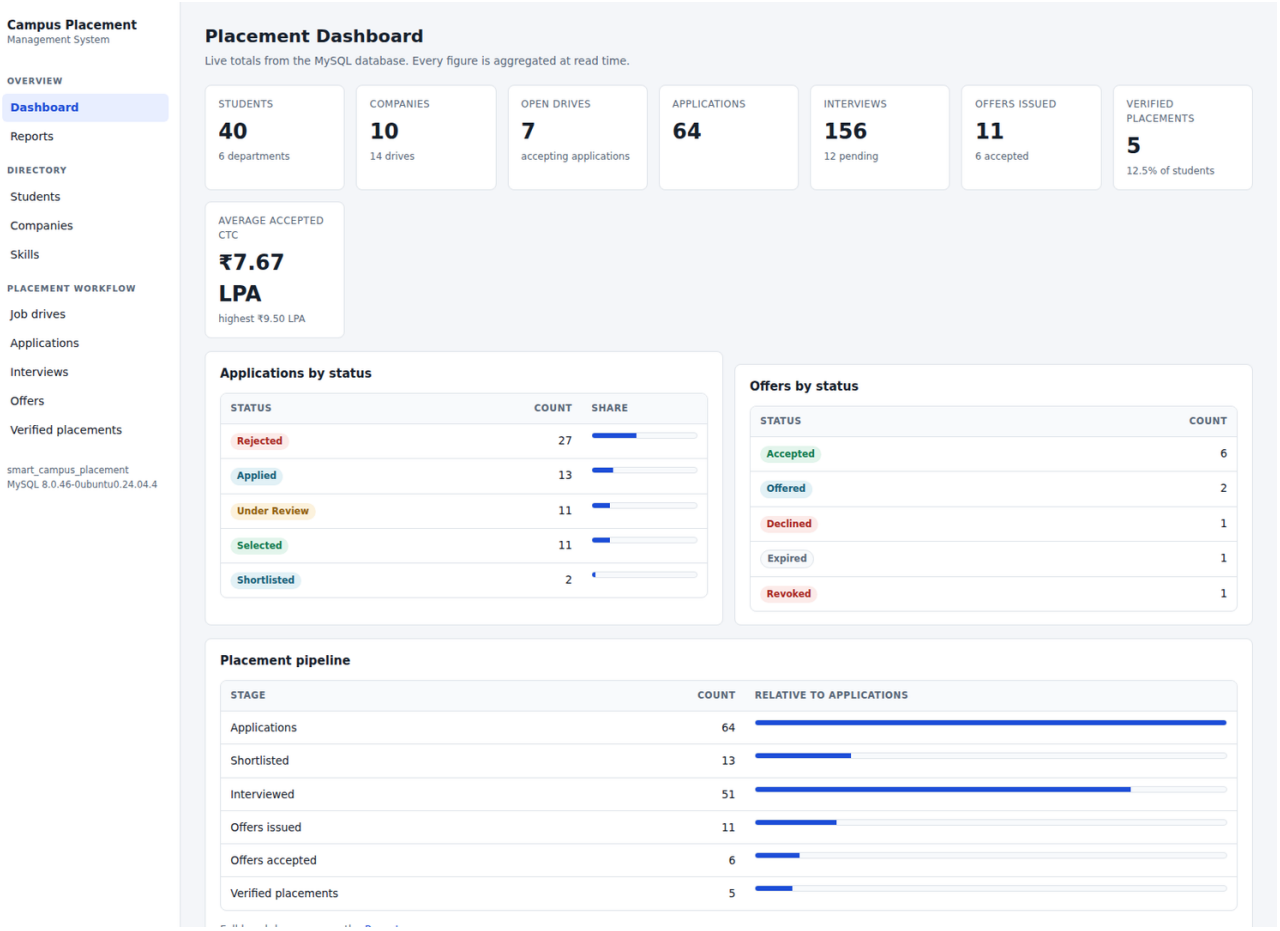


Figure 1. Dashboard. Every total is aggregated from the database at read time; none is stored. The footer reports the live MySQL version and database name.

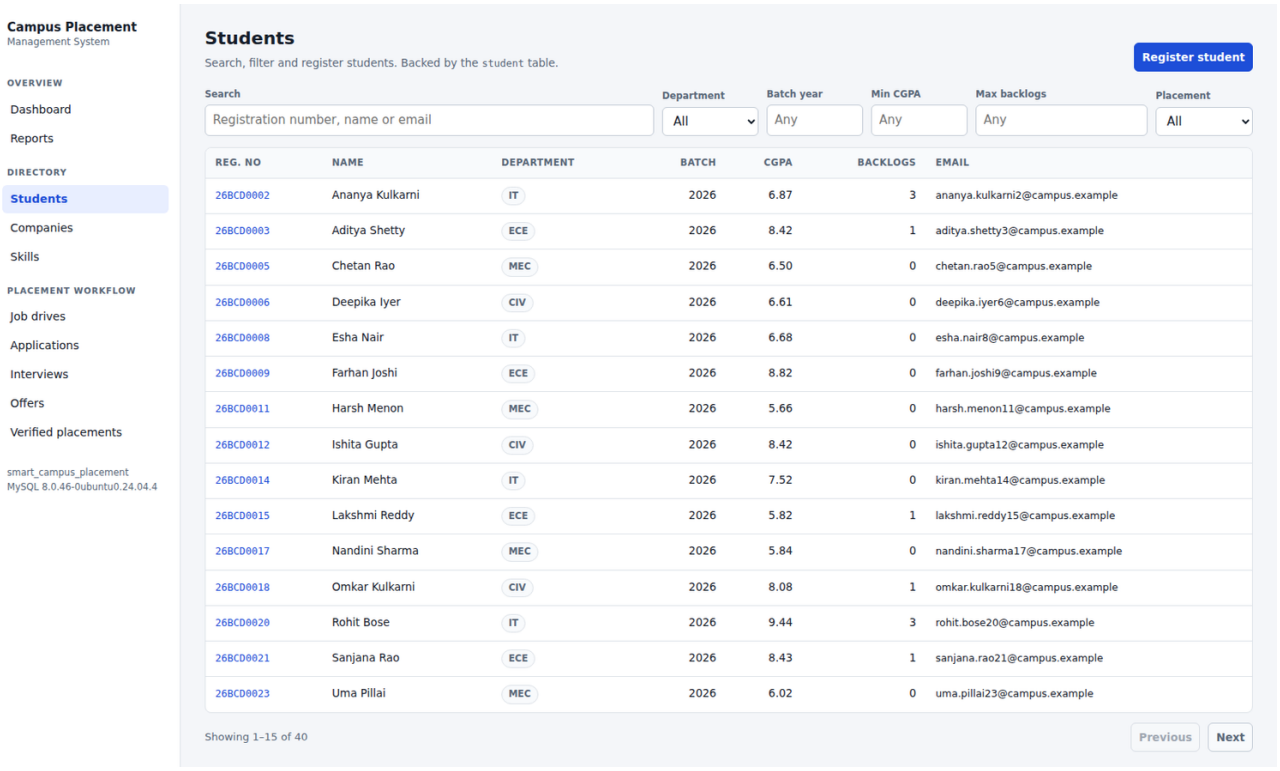


Figure 2. Student directory with search, six filters and server side pagination, showing 15 of 40 seeded students.

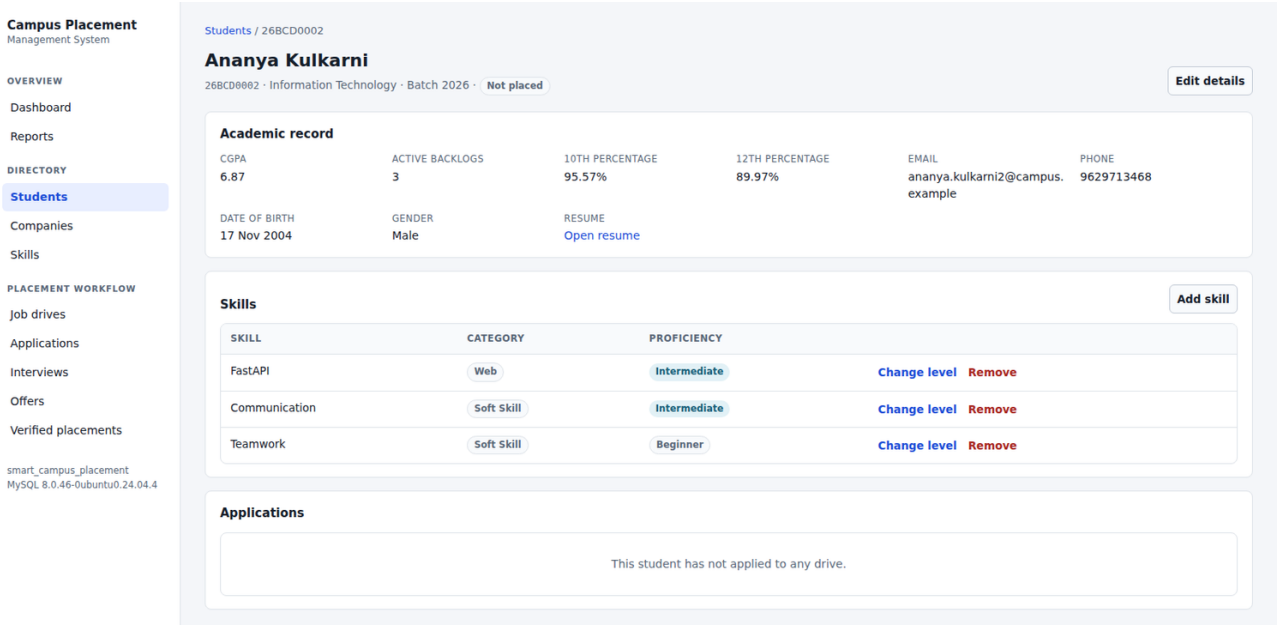


Figure 3. Student detail: academic record, skills drawn from the student_skill bridge relation, every application with its outcome, and the placement state.

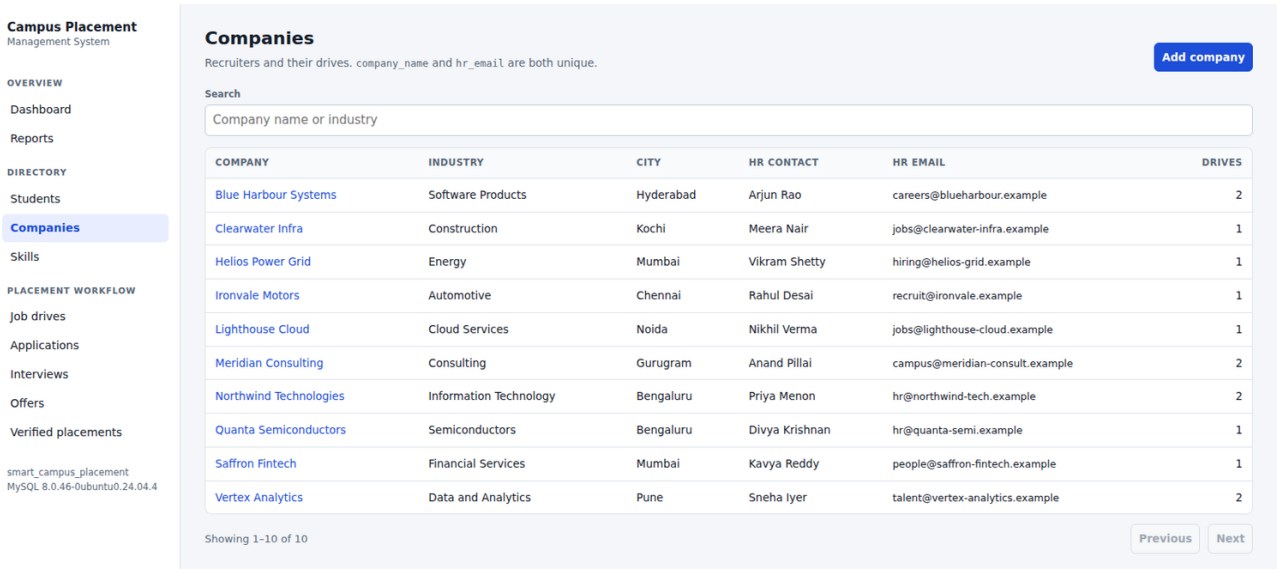


Figure 4. Company directory. company_name and hr_email are both UNIQUE in the Phase 1 schema, which the interface states explicitly.

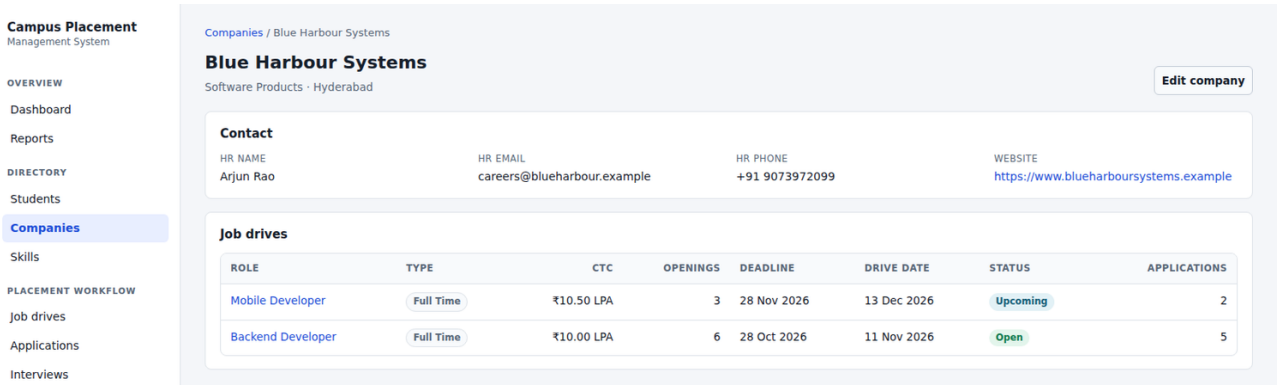


Figure 5. Company detail with its job drives and their application counts.

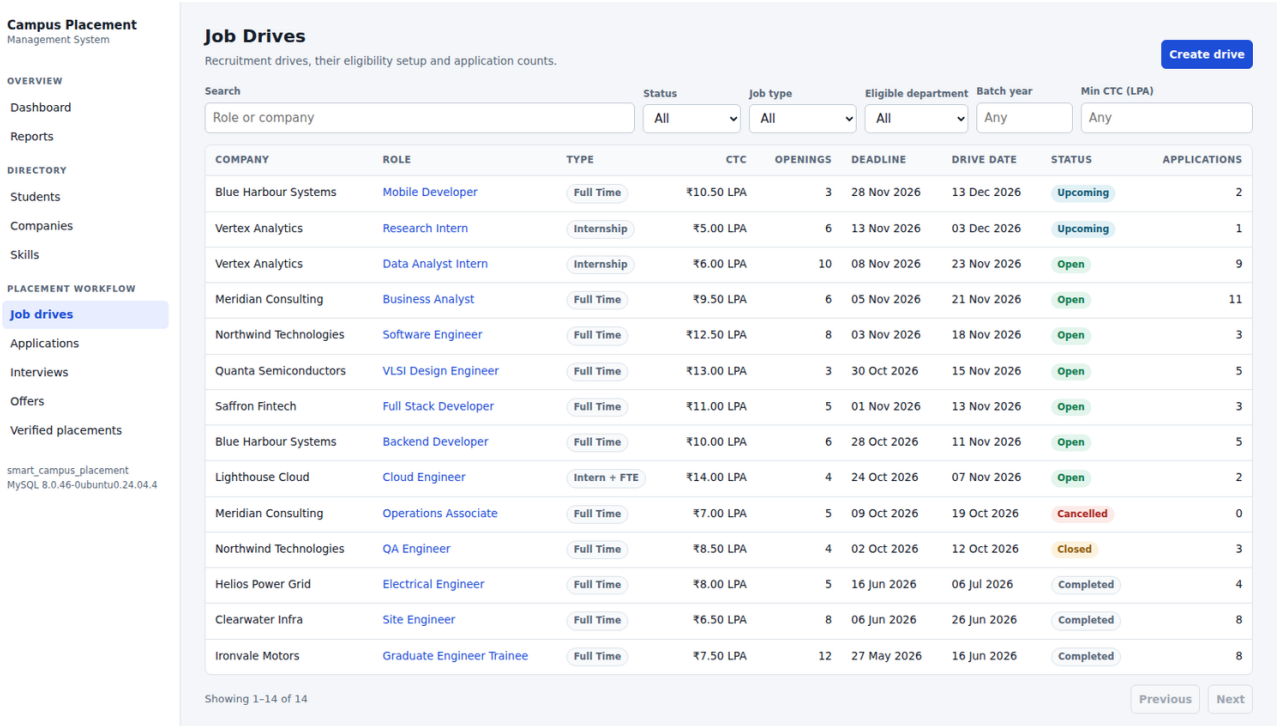


Figure 6. Job drive list with six filters, covering all five drive statuses.

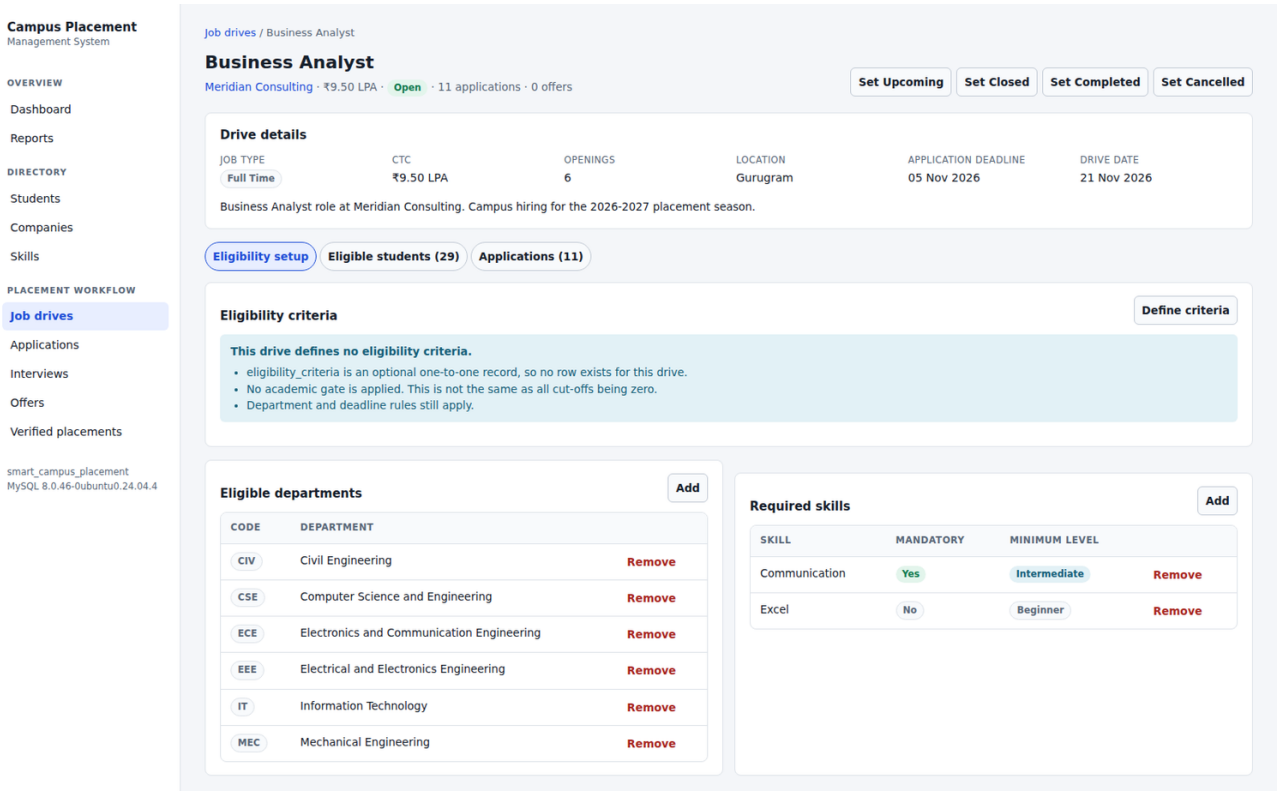


Figure 7. Drive detail. This drive has no eligibility_criteria row, and that condition is stated in plain language rather than shown as an empty section or as zero cut-offs. The eligible departments and required skills are also shown.

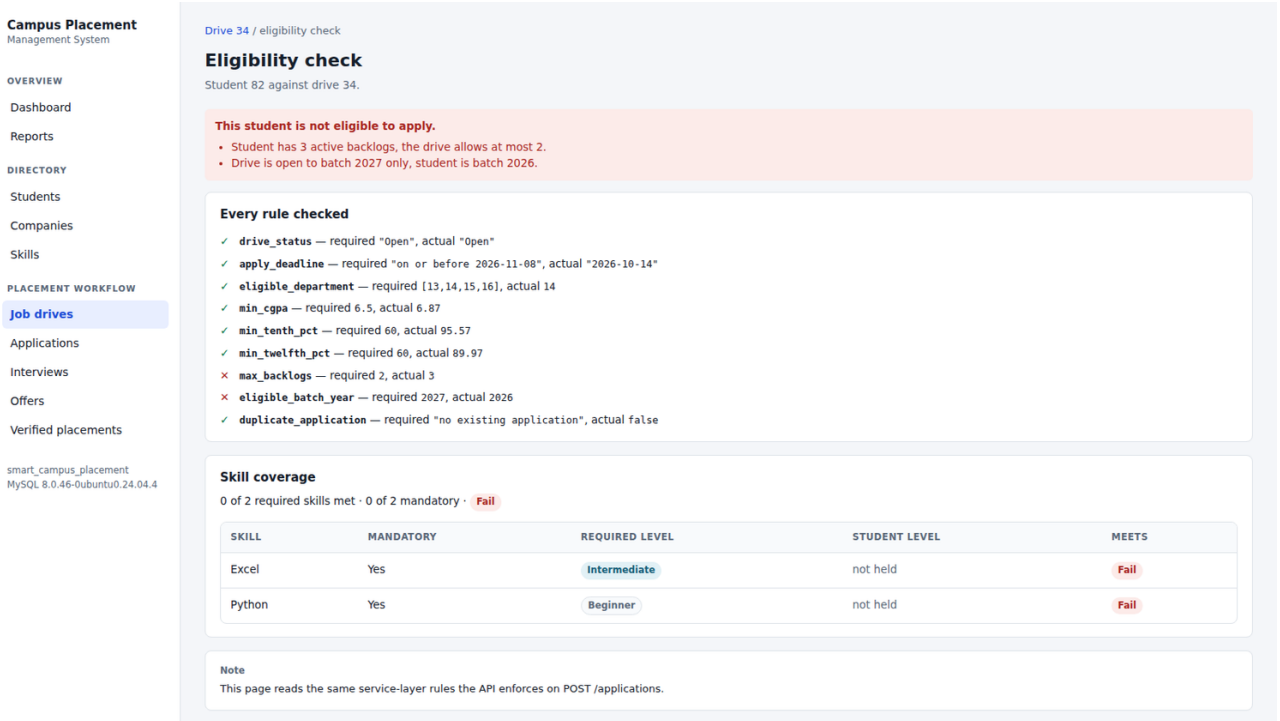


Figure 8. Eligibility evaluation for one student against one drive. Every rule is listed with its required and actual value; two have failed, and the reasons are given in words. This is the same code path that POST /api/applications calls before it writes.

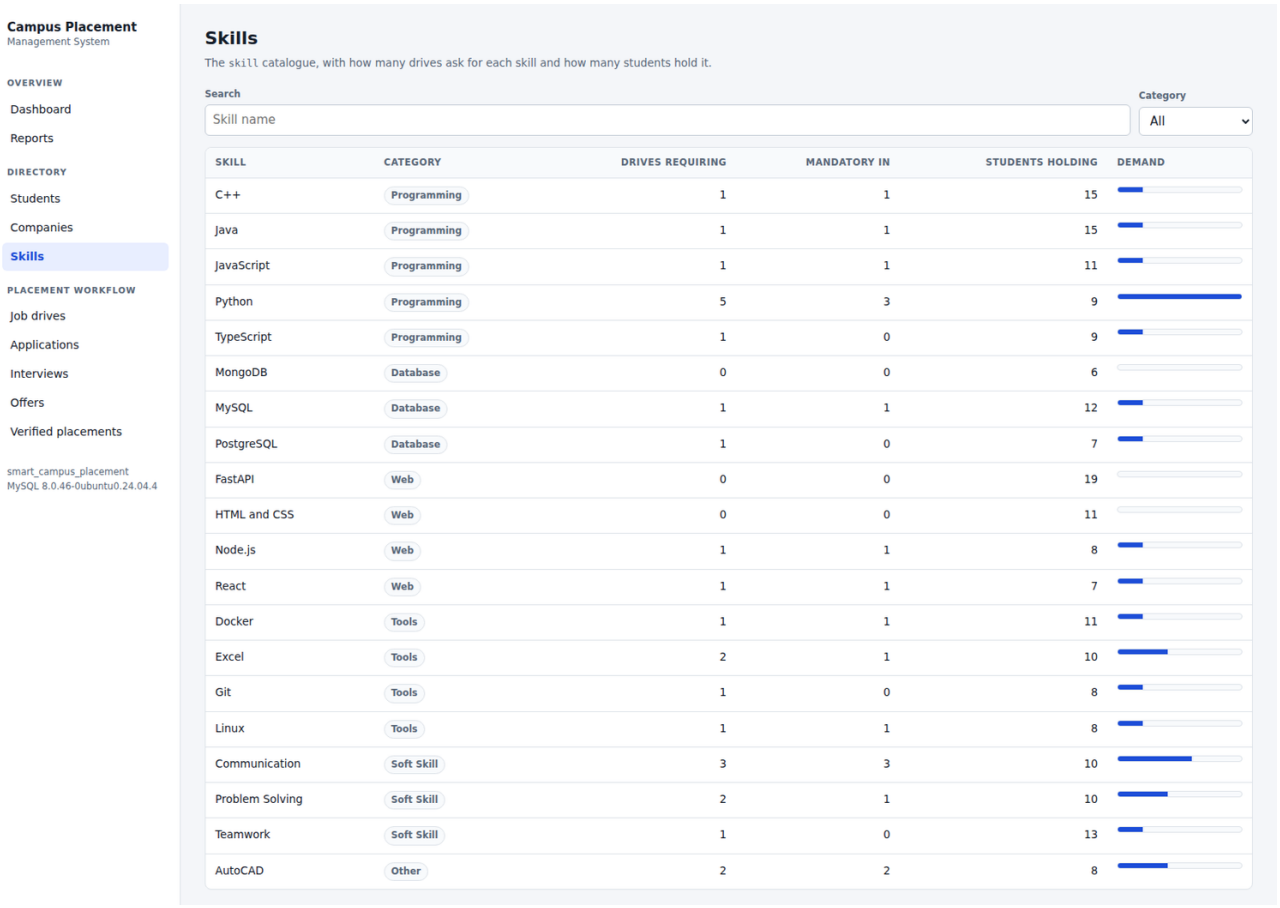


Figure 9. Skill catalogue, showing how many drives require each skill against how many students hold it.

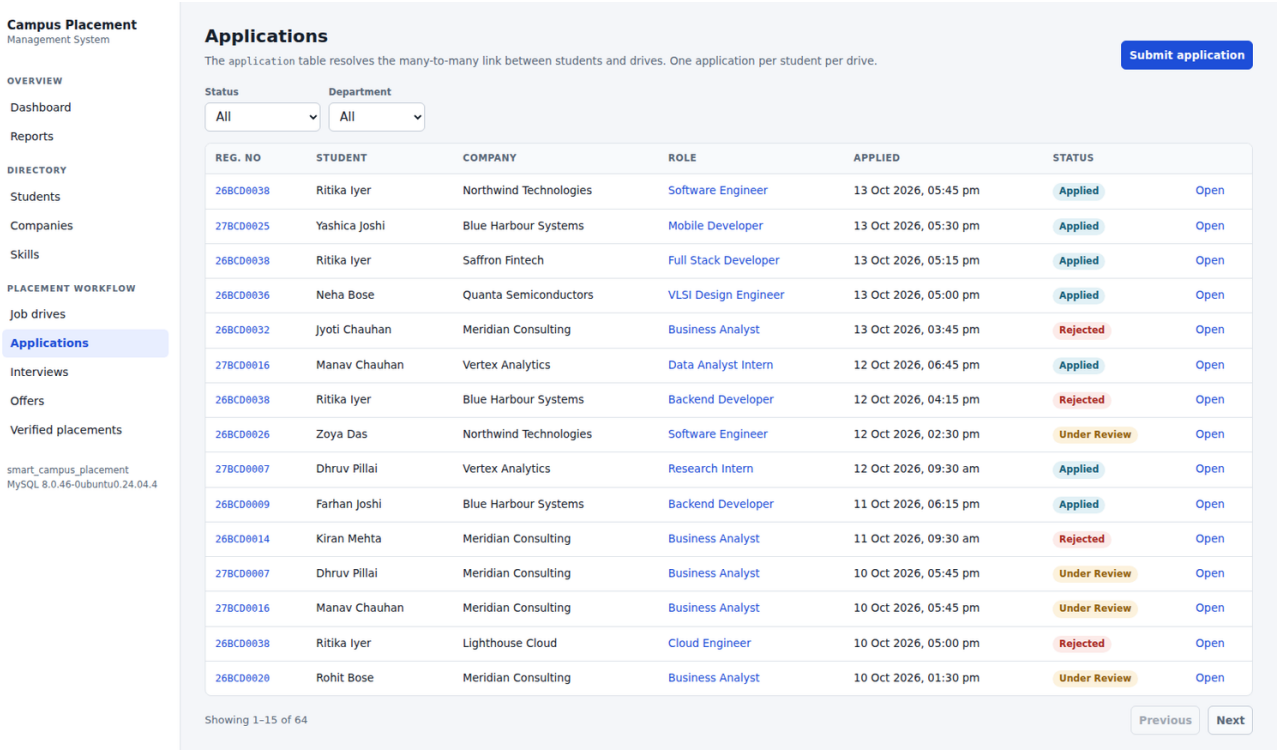


Figure 10. Application list, resolving the many to many relationship between students and drives, with status filters.

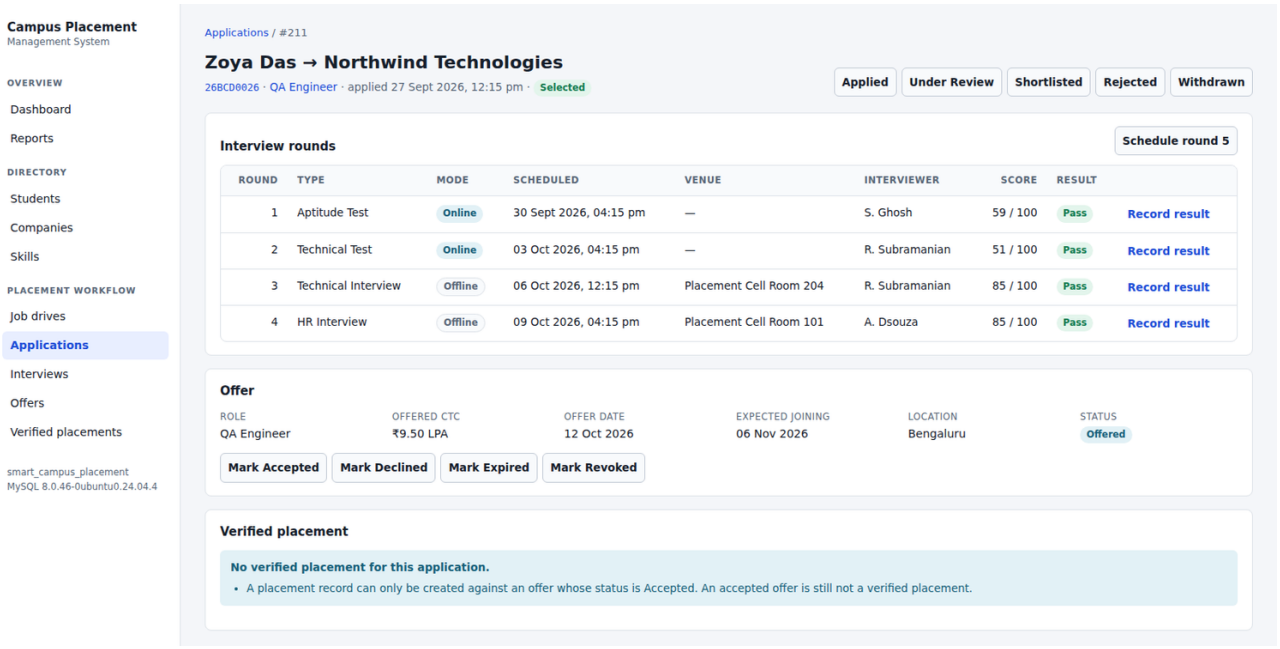


Figure 11. Application detail: four interview rounds with scores and results, the single offer permitted for this application, and the placement section explaining that an accepted offer is still not a verified placement.

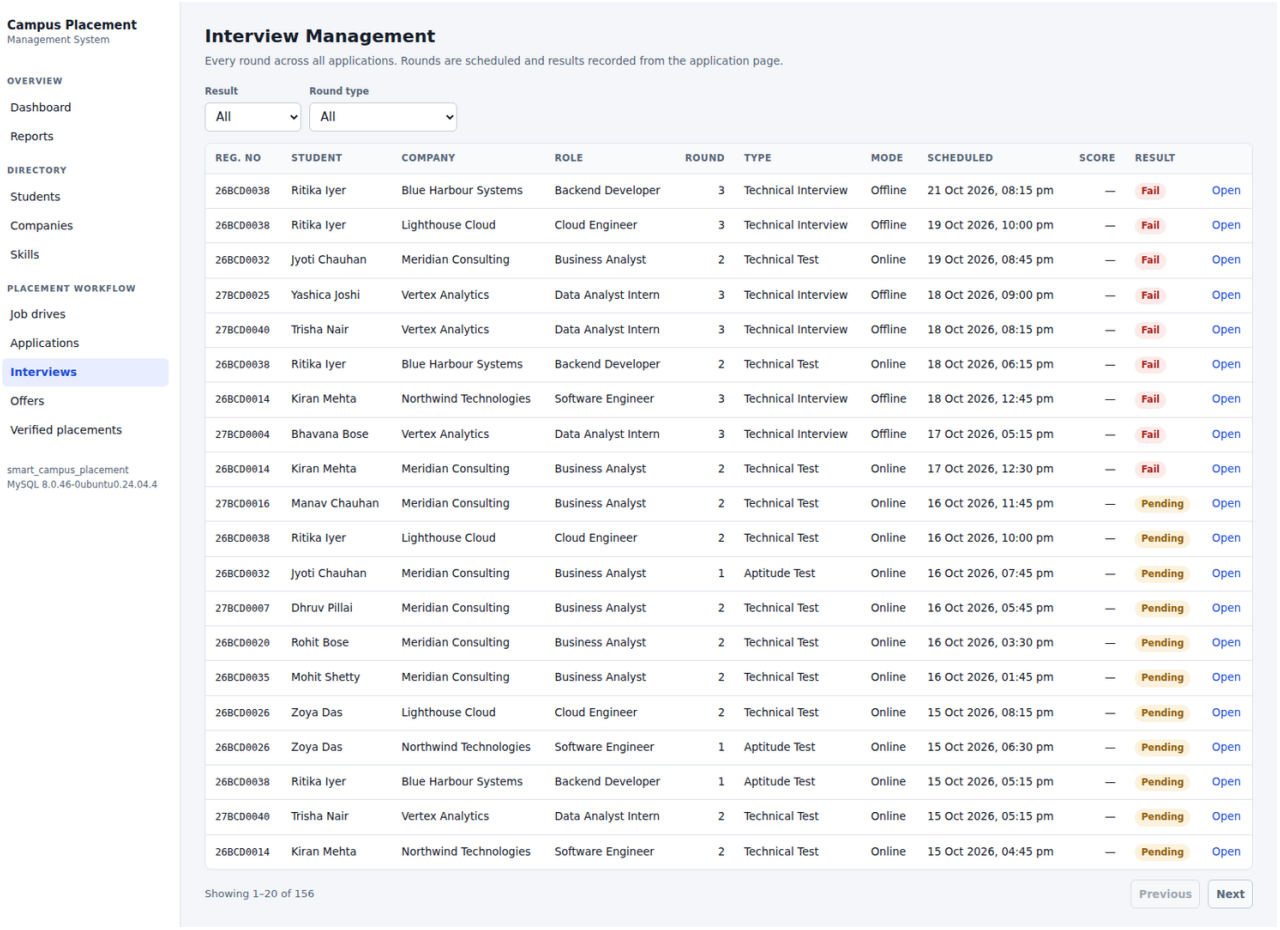


Figure 12. All interview rounds across every application, with result and round type filters.

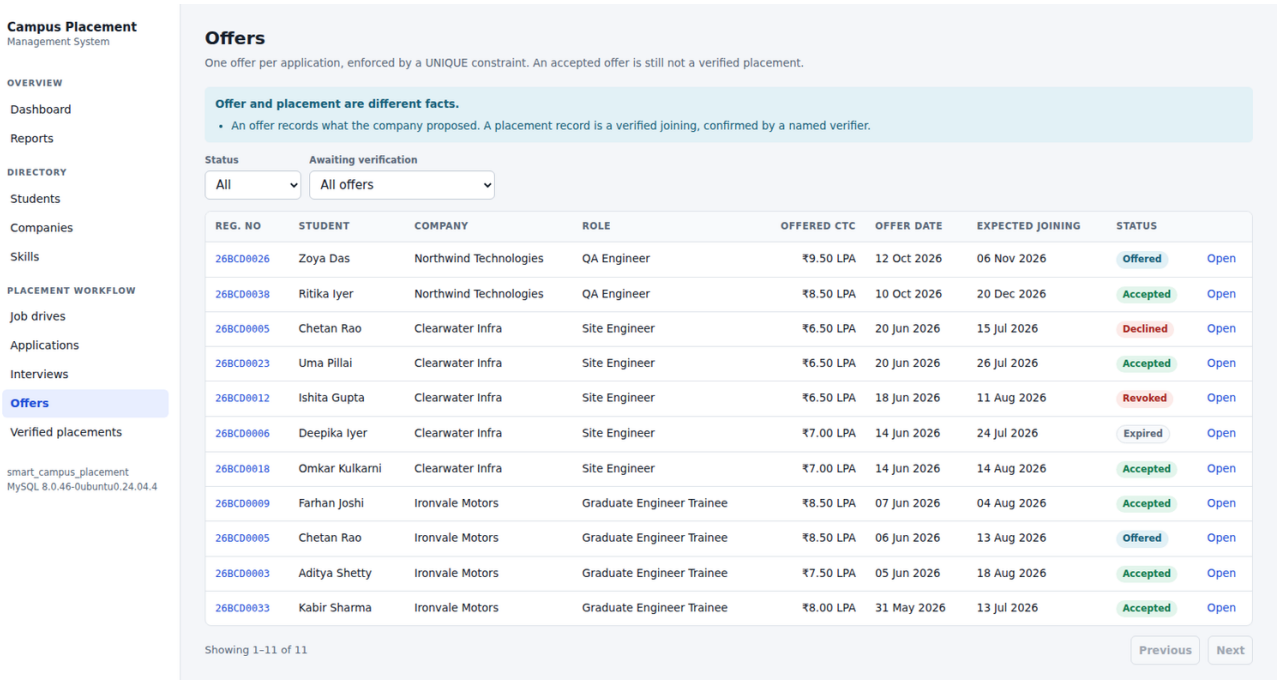


Figure 13. Offers, including the filter for accepted offers that have no verified placement record yet.

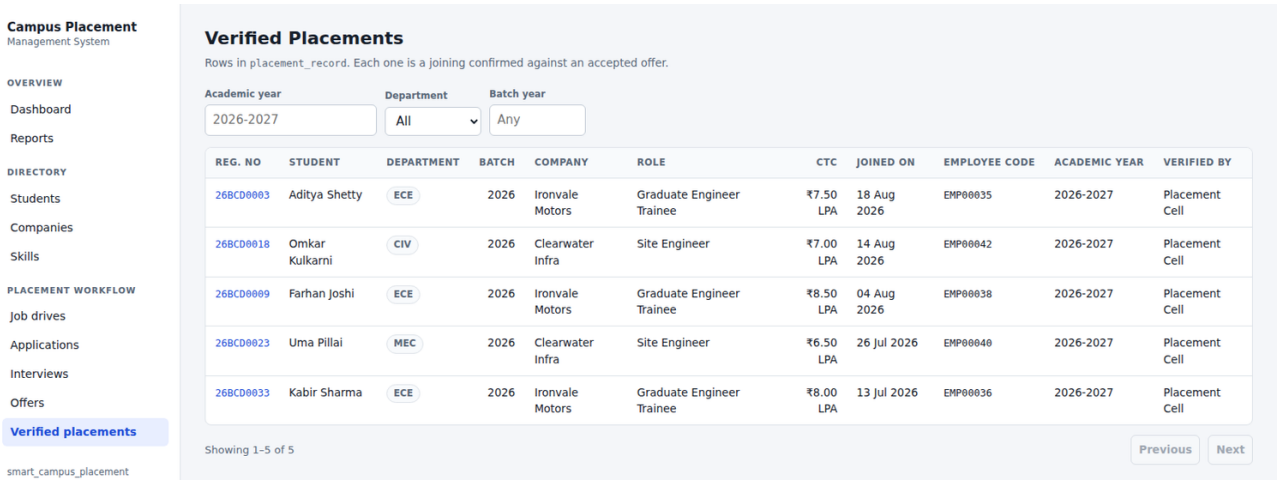


Figure 14. Verified placements, filtered by academic year, department and batch.

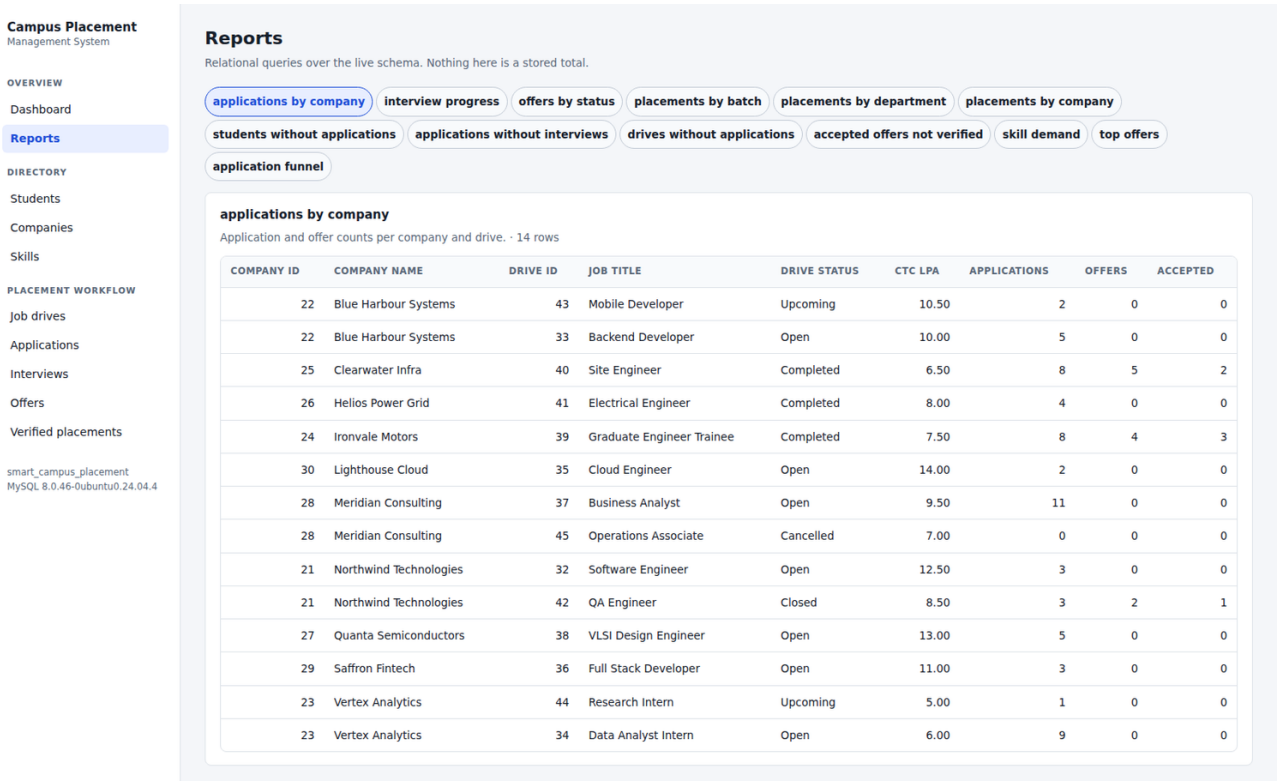


Figure 15. The report runner, executing applications-by-company over the live schema. All thirteen reports are reachable from the same screen.

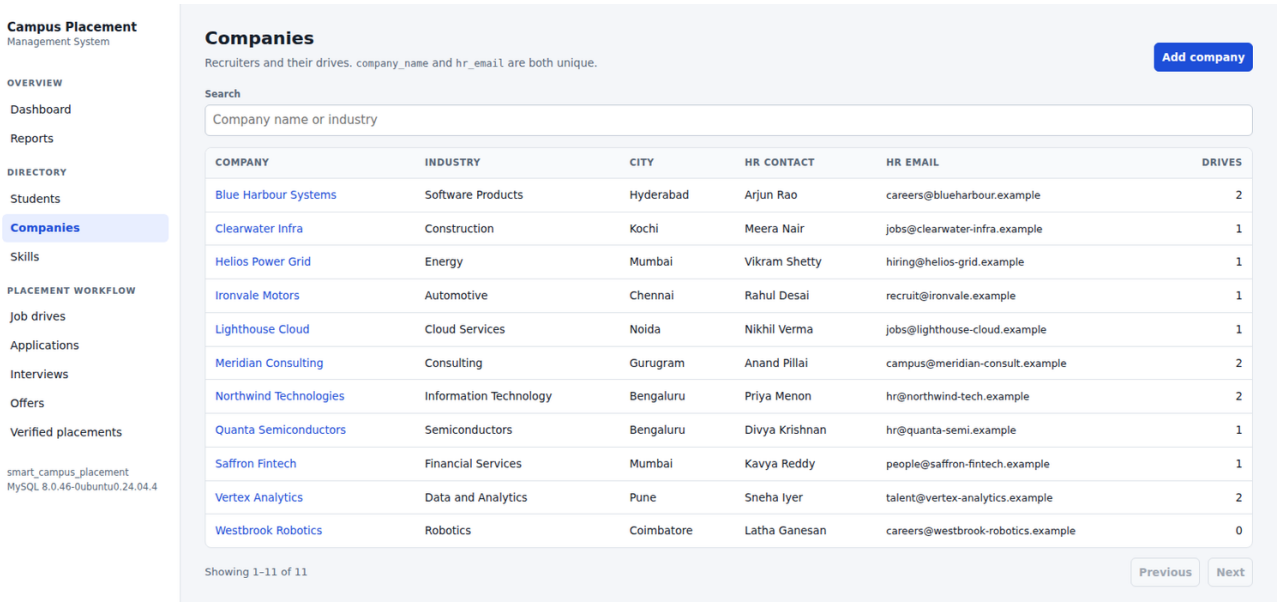


Figure 16. Write path verification. A company created through the browser's own network path appears in the interface, 'Showing 1-11 of 11'. The row was confirmed in MySQL and then deleted, so the demonstration data matches the seed.

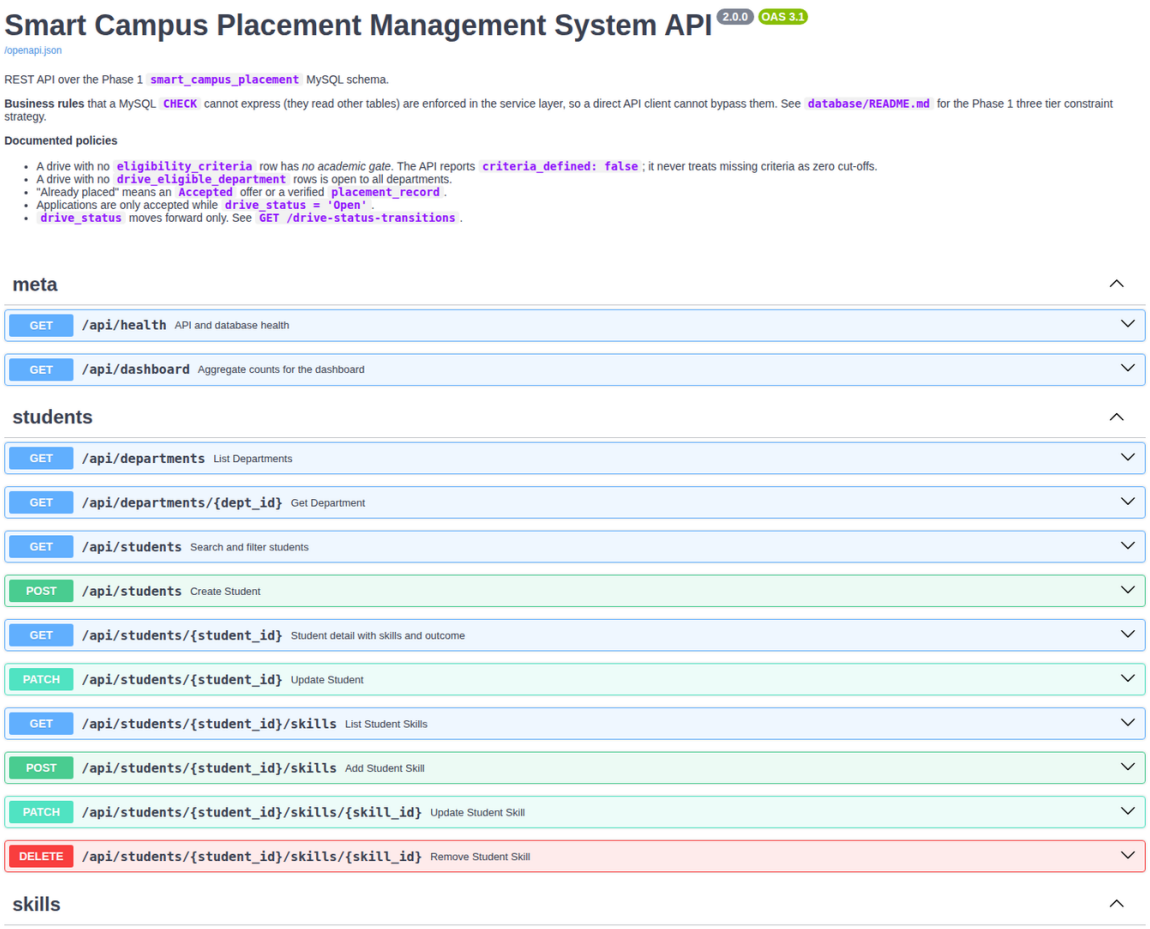


Figure 17. The OpenAPI documentation, generated from the code itself and served at /docs, so it cannot describe an endpoint that does not exist.

11. Limitations

The following are genuine shortcomings of the delivered system. They are listed in full rather than selectively.

1. One of the ten cross-table rules in section 6.1 is not implemented: company_employee_code is not checked for uniqueness within a company. That uniqueness is scoped to a company, which the present schema does not express, and implementing it would have required either a schema change or a cross-table check that no Phase 2 screen currently needs.

2. No automated browser test exists. The React event handlers were exercised only by hand. The endpoints behind every form are covered by the automated suite, and one write was verified through the browser's own network path, but no Playwright or Cypress suite clicks the buttons.

3. There is no authentication and no authorisation. Any caller can reach any endpoint. This is acceptable for a local academic build but would be unacceptable in deployment, where the placement officer, the student and the recruiter need distinct permissions.

4. No delete endpoint exists for the principal entities. Only the three bridge table links can be removed. The RESTRICT foreign keys make deleting a student or a company a policy question rather than a technical one, and that policy has not been decided.

5. The department and skill list endpoints are not paginated, because both are small reference lists. They would need pagination at a larger scale.

6. Offer expiry is not automatic. An offer whose date has passed remains in the Offered status until somebody changes it. A scheduled task would be required.

7. The dashboard recomputes every total on each request. This is correct, and fast at the present data volume, but would need caching at a much larger scale.

8. Interview rounds are not required to be contiguous. Round 3 can be created without round 2 existing. The schema permits this and no rule currently forbids it.

9. Concurrent load and large data volumes were not tested. Correctness under concurrency relies on the database UNIQUE constraints, which make the relevant races impossible, but no load test was performed.

12. Conclusion

Phase 2 delivers a working application on top of the Phase 1 database without altering it. The schema that was designed, normalised and created in Phase 1 is byte for byte the same schema the application runs against: thirteen tables, fourteen foreign keys, twenty-one CHECK constraints, no views and no triggers, verified after implementation.

The rules that Phase 1 could only document are now enforced. Nine of the ten cross-table rules are implemented in the service layer, where they apply to every caller rather than only to the browser, and each is covered by an automated test. The five questions Phase 1 left open have been decided explicitly, reported through the API and shown in the interface, rather than resolved silently inside the code.

The normalisation work has been honoured rather than quietly undone. No derived value is stored anywhere; placement counts, applicant totals and placement status are all computed at read time, and two automated tests demonstrate that they are derived rather than cached.

The system is verifiable. Sixty-seven integration tests run against the real database and all pass, each inside a transaction that is rolled back so the demonstration data is never disturbed. Fifty-two endpoints were exercised by hand in addition. Seventeen figures were captured from the running application. Four defects found during verification were corrected, and both the one outstanding rule and the absence of browser automation are disclosed rather than concealed.

What remains is principally access control and browser level testing, which together form the natural content of Phase 3.

13. Further Work

1. Role based access control, giving the placement officer, the student and the recruiter distinct views over the same API.

2. A browser end to end test suite covering the form interactions that the present suite reaches only through their endpoints.

3. Student self service: profile editing, resume upload and application tracking.

4. Notifications when a drive opens, a round is scheduled or an offer is issued.

5. Export of the reports to CSV or PDF for the placement office.

6. An audit trail of status changes. This would require a new table and is therefore an explicit schema decision for a later phase rather than a silent addition in this one.

7. Deployment packaging, with a production configuration and a proper secret store.

Appendix A. API Endpoint Reference

Fifty-two endpoints, listed as the generated OpenAPI document reports them. The base path is /api.

Method	Path	Description
GET	/health	API and database health, reported separately
GET	/dashboard	Aggregate counts for the dashboard
GET	/departments	List departments
GET	/departments/{dept_id}	Department detail
GET	/students	Search and filter students
POST	/students	Create a student
GET	/students/{student_id}	Student detail with skills and outcome
PATCH	/students/{student_id}	Update a student
GET	/students/{student_id}/skills	List a student's skills
POST	/students/{student_id}/skills	Assign a skill to a student
PATCH	/students/{student_id}/skills/{skill_id}	Update proficiency
DELETE	/students/{student_id}/skills/{skill_id}	Remove a skill assignment
GET	/skills	List skills
GET	/skills/{skill_id}	Skill detail
GET	/companies	List companies
POST	/companies	Create a company
GET	/companies/{company_id}	Company with its drives
PATCH	/companies/{company_id}	Update a company
GET	/drives	Search and filter drives
POST	/drives	Create a drive
GET	/drives/{drive_id}	Drive detail with eligibility setup
PATCH	/drives/{drive_id}	Update a drive
GET	/drive-status-transitions	The documented status transition policy
GET	/drives/{drive_id}/eligibility-criteria	Criteria row, or null when none is defined
PUT	/drives/{drive_id}/eligibility-criteria	Create or replace the criteria
POST	/drives/{drive_id}/departments/{dept_id}	Add an eligible department
DELETE	/drives/{drive_id}/departments/{dept_id}	Remove an eligible department
POST	/drives/{drive_id}/skills	Add a required skill
DELETE	/drives/{drive_id}/skills/{skill_id}	Remove a required skill
GET	/drives/{drive_id}/eligibility/{student_id}	Why one student is or is not eligible
GET	/drives/{drive_id}/eligible-students	Students satisfying the drive's criteria
GET	/drives/{drive_id}/skill-match/{student_id}	Skill coverage for one student
GET	/applications	List and filter applications
POST	/applications	Submit an application with full rule checking
GET	/applications/{application_id}	Application with interviews and offer
PATCH	/applications/{application_id}	Update the application status
GET	/applications/{application_id}/eligibility	Re-run the eligibility report

Method	Path	Description
GET	/applications/{application_id}/offer	The one offer for an application
GET	/interviews	List and filter interview rounds
POST	/interviews	Schedule an interview round
GET	/interviews/{interview_id}	Interview detail
PATCH	/interviews/{interview_id}	Record schedule, score, result and feedback
GET	/offers	List and filter offers
POST	/offers	Issue an offer
GET	/offers/{offer_id}	Offer detail
PATCH	/offers/{offer_id}	Update an offer
GET	/placements	List and filter verified placements
POST	/placements	Record a verified joining
GET	/placements/{placement_id}	Placement detail
PATCH	/placements/{placement_id}	Update a placement record
GET	/reports	List the available reports
GET	/reports/{name}	Run one named report

Appendix B. Selected Source Listings

B.1 Eligibility evaluation, from backend/app/rules.py

The department and academic checks, abridged. Each check records its required and actual value so the interface can explain a rejection rather than merely report one.

```
# Department restriction. No rows means every department may apply.
dept_ids = eligible_department_ids(conn, drive_id)
department_restricted = bool(dept_ids)
if department_restricted:
    ok = int(student["dept_id"]) in dept_ids
    checks.append(_check("eligible_department", dept_ids, student["dept_id"], ok))
    if not ok:
        reasons.append(
            "Student's department is not in the drive's eligible department list.")

# Academic criteria. No row means no academic gate, not zero cut-offs.
criteria = get_by_pk(conn, "eligibility_criteria", "drive_id", drive_id)
criteria_defined = criteria is not None

if criteria_defined:
    ok = int(student["active_backlogs"]) <= int(criteria["max_backlogs"])
    checks.append(_check("max_backlogs", criteria["max_backlogs"],
                         student["active_backlogs"], ok))
    if not ok:
        reasons.append(
            f"Student has {student['active_backlogs']} active backlogs, the "
            f"drive allows at most {criteria['max_backlogs']}.")
    ...
else:
    # Reported as undefined, never silently treated as zero cut-offs.
    checks.append(_check("eligibility_criteria", "optional",
                         "not defined for this drive", True))
```

B.2 The placement rule, from backend/app/rules.py

A placement record may be created only against an accepted offer. The uniqueness of the placement is additionally guaranteed by uq_placement_offer.

```
def assert_placement_ok(conn, offer_id):
    offer = require(conn, "offer", "offer_id", offer_id)
    # A placement may only be recorded against an accepted offer.
    if offer["offer_status"] != "Accepted":
        raise RuleViolation(
            f"Offer {offer_id} has status '{offer['offer_status']}'. A placement "
            "record may only be created against an 'Accepted' offer.")
    # Also guarded by uq_placement_offer.
    if exists(conn, "placement_record", offer_id=offer_id):
        raise RuleViolation(
            f"Offer {offer_id} already has a verified placement record.")
    return offer
```

B.3 A reporting query, from backend/app/routers/reports.py

Placement count and placement rate per department. NULLIF prevents division by zero for a department with no students.

```
SELECT  d.dept_id,
        d.dept_code,
```

```
        d.dept_name,
        COUNT(DISTINCT s.student_id)    AS total_students,
        COUNT(DISTINCT pr.placement_id) AS placements,
        ROUND(100 * COUNT(DISTINCT pr.placement_id)
                  / NULLIF(COUNT(DISTINCT s.student_id), 0), 2) AS placement_pct,
        ROUND(AVG(o.offered_ctc_lpa), 2) AS avg_ctc_lpa
FROM        department d
LEFT JOIN   student     s  ON s.dept_id = d.dept_id
LEFT JOIN   application a  ON a.student_id = s.student_id
LEFT JOIN   offer       o  ON o.application_id = a.application_id
LEFT JOIN   placement_record pr ON pr.offer_id = o.offer_id
GROUP BY    d.dept_id
ORDER BY    placements DESC, d.dept_code
```

B.4 Transaction scope, from backend/app/db.py

```
def get_conn():
    """One connection and one transaction per request.

    The route handler runs inside conn.begin(). Any exception rolls the
    whole request back, so multi-write operations are atomic by default.
    """
    with engine.connect() as conn:
        with conn.begin():
            yield conn
```

B.5 Running the system

```
# Backend
cd backend
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
cp .env.example .env            # set DB_USER, and DB_PASSWORD if one is used
.venv/bin/python seed.py        # load the demonstration data
.venv/bin/uvicorn app.main:app --reload --port 8000

# Frontend
cd frontend
npm install
npm run dev

# Tests
cd backend && .venv/bin/python -m pytest -v

# Application   http://127.0.0.1:5173
# API           http://127.0.0.1:8000/api
# API docs      http://127.0.0.1:8000/docs
```